Looking at our 2D example, where the cost function depends on a single variable, it seems pretty obvious the direction we should explore!

But in a multidimensional space, where we can't visualize the shape of the surface, how do we know which parameters should be tuned and in which direction?

The mathematical solution to this quest is called **gradient descent**. Gradient descent is an optimization technique that, under certain conditions, can be applied to different categories of optimization problems.

The idea is simple: we can look at the slope of a function at a given point, and move towards the direction the function decreases the fastest. The name stems from the fact that, for a differentiable function, the direction of the steepest slope is given by its gradient.

This is briefly illustrated in figure 16.13 for a single-variable, differentiable function; before discussing it a bit more formally, let's see how we could frame it in our marbles example: it's like we are allowed to push marbles with a nudge, and as such they can travel a short distance before stopping again in the sand. Similarly to what happens when you play marbles in the sand, you several nudges to reach the goal, and they only move for a short distance; if you give them a nudge in the right direction, they move a little further and toward the finish line (or your goal). But to make things more interesting, in our game you can only see a short portion of the track next to your current position: as if you were playing in the fog, or with lens distortion.

Gradient descent is formally described using *calculus*: don't worry if you haven't had an introduction to calculus, you are not going to need it to apply gradient descent, there are many great libraries that already implement it for you: actually, thinking